

ПЗ №15. Изучение пакетных менеджеров Python

Знакомство с инструментами управления зависимостями в Python: стандартный пакетный менеджер `pip` и современный высокопроизводительный менеджер `uv`. Работа с виртуальным окружением, установка сторонних библиотек и сохранение зависимостей.

Цель работы

- **Теоретическая:** Изучить назначение и принципы работы пакетных менеджеров, виртуальных окружений и файла `requirements.txt`.
- **Практическая:** Приобрести навыки создания виртуального окружения, установки библиотеки `requests`, написания программы для HTTP-запросов к публичному API, фиксации зависимостей через `pip freeze`. Освоить альтернативный менеджер `uv` и сравнить его с `pip`.

Задачи

1. Создать виртуальное окружение с помощью `python -m venv`.
2. Установить библиотеку `requests` через `pip`.
3. Написать программу, которая обращается к общедоступному REST API (JSONPlaceholder) и выводит полученные данные.
4. Сформировать файл `requirements.txt` командой `pip freeze`.
5. Повторить все шаги, используя пакетный менеджер `uv` (установка `uv`, создание окружения, установка `requests`, запуск программы, экспорт зависимостей).
6. Сравнить два подхода.

Оборудование и программное обеспечение

- **Программное обеспечение:** Python 3.8+, интерпретатор командной строки (терминал Linux / PowerShell или CMD в Windows), текстовый редактор (VS Code).
- **Оборудование:** Персональный компьютер с ОС Linux (Manjaro) или Windows 10, доступ в интернет.

Краткие теоретические сведения

Пакетный менеджер – инструмент для автоматической установки, обновления и удаления сторонних библиотек (пакетов) Python. Стандартным менеджером

является `pip`. Он работает с репозиторием PyPI (Python Package Index).

Виртуальное окружение – изолированная среда, в которой можно устанавливать библиотеки без конфликта с глобальными пакетами системы. Создаётся командой `python -m venv <имя_окружения>`. После активации все `pip install` будут ставить пакеты только внутрь этого окружения.

Файл `requirements.txt` – текстовый файл со списком зависимостей проекта и их версий. Создаётся командой `pip freeze > requirements.txt`. Позволяет воспроизвести окружение на другой машине командой `pip install -r requirements.txt`.

Библиотека `requests` – популярный HTTP-клиент для Python, упрощающий отправку запросов к веб-серверам. Устанавливается как `pip install requests`.

Пакетный менеджер `uv` – альтернатива `pip`, написанная на Rust, значительно быстрее (`uv pip install ...`). Поддерживает совместимость с `pip` и файлами `requirements.txt`. Устанавливается отдельно.

Публичное API – в работе используется JSONPlaceholder (<https://jsonplaceholder.typicode.com>) – фиктивный REST API для тестирования. Эндпоинт `/posts/1` возвращает данные поста в формате JSON.

Порядок выполнения работы

1. Подготовительный этап

1. Откройте терминал (Linux: `Ctrl+Alt+T`; Windows: `Win+R` → `cmd` или PowerShell).
2. Создайте рабочую папку для лабораторной работы, например `lab15_packages`.

```
mkdir lab15_packages
cd lab15_packages
```

3. Убедитесь, что Python установлен: `python --version` (или `python3` в Linux). При необходимости установите Python.

2. Основной этап

Задание 1. Работа с `pip` и стандартным виртуальным окружением

1. Создайте виртуальное окружение командой:

```
python -m venv venv
```

(В Linux/Mac возможно потребуется `python3 -m venv venv`)

2. Активируйте окружение:

- **Windows (CMD):** `venv\Scripts\activate.bat`
- **Windows (PowerShell):** `venv\Scripts\Activate.ps1` (если разрешено выполнение скриптов)
- **Linux / Manjaro:** `source venv/bin/activate`

После активации в начале строки терминала появится `(venv)` .

3. Установите библиотеку `requests` :

```
pip install requests
```

4. Напишите программу `api_client.py` в той же папке (с помощью VS Code или любого редактора). Код программы:

```
import requests

def fetch_post(post_id):
    url = f"https://jsonplaceholder.typicode.com/posts/{post_id}"
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    else:
        return None

if __name__ == "__main__":
    post = fetch_post(1)
    if post:
        print("Полученные данные:")
        print(f"ID: {post['id']}")
        print(f"Заголовок: {post['title']}")
        print(f"Содержимое: {post['body']}")
    else:
        print("Ошибка при получении данных")
```

5. Запустите программу:

```
python api_client.py
```

Вы должны увидеть вывод с данными поста.

6. Создайте файл `requirements.txt` :

```
pip freeze > requirements.txt
```

Откройте `requirements.txt` – внутри должно быть что-то вроде `requests==2.31.0` и возможно зависимости.

7. Деактивируйте окружение:

```
deactivate
```

Задание 2. Работа с пакетным менеджером `uv`

1. Установите `uv` (если ещё не установлен):

- **Linux (Manjaro):** через официальный установщик:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

(или через `pip install uv`, но рекомендуется отдельная установка)

- **Windows:** в PowerShell (администратор):

```
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

После установки перезапустите терминал или выполните `source ~/.bashrc` (Linux).

2. Создайте новое виртуальное окружение с `uv` (можно в той же папке, но лучше создать подпапку `uv_demo`):

```
mkdir uv_demo
cd uv_demo
uv venv
```

Будет создано окружение в папке `.venv`.

3. Активируйте окружение (теперь оно в `.venv`):

- **Linux:** `source .venv/bin/activate`
- **Windows CMD:** `.venv\Scripts\activate.bat`
- **Windows PowerShell:** `.venv\Scripts\Activate.ps1`

4. Установите `requests` с помощью `uv pip`:

```
uv pip install requests
```

(Обратите внимание на скорость установки)

5. Создайте такую же программу `api_client.py` (можно скопировать из предыдущей папки).

6. Запустите программу:

```
python api_client.py
```

7. Экспортируйте зависимости (аналог `pip freeze`):

```
uv pip freeze > requirements_uv.txt
```

Сравните содержимое `requirements_uv.txt` с предыдущим `requirements.txt` – они должны быть одинаковыми (или почти одинаковыми).

8. Деактивируйте окружение:

```
deactivate
```

Задание 3. Сравнение

- Сравните время установки `requests` через `pip` и `uv`.
- Сравните размер созданных папок `venv` и `.venv`.
- Отметьте удобство команд.

3. Контрольный этап

1. Убедитесь, что программа корректно выполняется в обоих окружениях.
2. Проверьте, что файлы `requirements.txt` и `requirements_uv.txt` созданы и содержат записи о зависимостях.
3. Убедитесь, что после деактивации окружения команда `pip list` показывает только глобальные пакеты (без `requests`).

Контрольные вопросы

1. Зачем нужны виртуальные окружения в Python?
2. Что такое `pip` и для чего используется команда `pip freeze`?
3. Какие преимущества даёт использование файла `requirements.txt`?
4. Как активировать виртуальное окружение в Windows и Linux?
5. Что делает библиотека `requests`? Приведите примеры других популярных библиотек.
6. Чем отличается `uv` от `pip`? В чём его главное преимущество?
7. Можно ли использовать `uv` без создания отдельного виртуального окружения?
8. Что произойдёт, если установить `requests` глобально без виртуального окружения?
9. Как удалить виртуальное окружение?
10. Как воспроизвести окружение на другом компьютере, имея только `requirements.txt`?

Содержание отчета

1. Тема, цель и задачи работы.
2. Листинг программы `api_client.py`.
3. Скриншоты или текст вывода программы в терминале (для обоих окружений).
4. Содержимое файлов `requirements.txt` и `requirements_uv.txt`.

5. Ответы на контрольные вопросы.

6. **Выводы по работе:** Сравнение `pip` и `uv`, какие трудности возникли при активации окружений на вашей ОС, почему важно изолировать зависимости проектов. Достигнута ли цель работы.